# Guía provisional: de Simulink a Arduino Uno emulado

**Mini-taller de generación automática de código para sistemas embebidos**

**Curso:** Taller de Sistemas Embebidos

## 1. Propósito y alcance de este avance

Esta guía explica cómo se preparó el entorno, se construyó un modelo Simulink, se generó y compiló su código y se ejecutó el firmware en un Arduino Uno R3 virtual. El ejercicio utiliza un potenciómetro para controlar el brillo de un LED.

El resultado alcanzado es un flujo funcional **modelo → código → firmware → emulación**. Todavía no está completo el flujo Model-Based DevOps propuesto: falta integrar la compilación y las verificaciones en GitHub Actions.

Hay dos papeles distintos:

- **Expositor:** prepara MATLAB, el modelo, el firmware y, posteriormente, CI.
- **Participante:** usa los archivos preparados y ejecuta el tutorial virtual en VS Code con Wokwi; no necesita MATLAB ni una placa física para esta parte.

La demostración con Arduino físico será una etapa posterior. La emulación no reemplaza esa validación ni demuestra por sí sola el comportamiento eléctrico del montaje real.

### Qué se comprobó y qué no

| Elemento | Estado al corte |
|---|---|
| MATLAB, Simulink y productos de generación | Instalados y reconocidos |
| Soporte Arduino y biblioteca de bloques | Instalados y cargados |
| Modelo editable y firmware Uno R3 | Generados y compilados |
| Compilación sin buscar una placa | Comprobada después de corregir el target |
| Circuito y rutas Wokwi | Verificados estáticamente |
| Brillo mínimo, intermedio, máximo, retorno y reinicio | Confirmados manualmente por el expositor |
| Frecuencia PWM, periodo de ejecución y máximo de RAM | No medidos |
| UART, analizador de señales y pruebas automáticas | Pendientes |
| GitHub Actions, demo física y materiales académicos finales | Pendientes |

**Trazabilidad:** la instalación inicial de MATLAB ya existía al comenzar. El paso 3 explica cómo preparar esa base en otro equipo, pero no se presenta como una instalación ejecutada durante esta sesión. La adición de Simulink, Coder y el soporte Arduino sí se realizó y verificó. El registro histórico está en bitacora.md.

# 2. Entorno y organización

## Versiones observadas

| Componente | Versión o configuración |
|---|---|
| Anfitrión | Linux x86_64 |
| MATLAB | R2026b, 26.2.0.3386108 |
| Simulink, Simulink Coder, Embedded Coder y MATLAB Coder | 26.2 |
| Simulink Support Package for Arduino Hardware | 26.2.3 |
| MATLAB Package Manager | 2026.7.1, glnxa64 |
| Wokwi para VS Code | 4.0.0 |
| Target | Arduino Uno R3, ATmega328P a 16 MHz |
| Toolchain AVR del soporte | 7.3.0-atmel3.6.1-arduino7 |
| Core Arduino AVR | 1.8.3 |

Estas son las versiones probadas, no una afirmación de compatibilidad con todas las versiones desde R2020b. Para otra versión o sistema operativo se deben revisar soporte, rutas, APIs y resultados.

## Requisitos de acceso

- Expositor: acceso válido a los productos MathWorks utilizados, Internet, espacio de instalación y permisos de escritura en su carpeta de usuario.
- Participante: VS Code, extensión Wokwi, Internet y activación válida de Wokwi.
- No se presupone que crear una cuenta gratuita otorgue una licencia permanente para Wokwi en VS Code. Verificar el acceso de cada participante antes de clase.
- No introducir contraseñas, claves de activación o tokens en el repositorio.

El entorno tenía 167 GB libres al comenzar; es una observación, no el requisito mínimo del proyecto. Los requisitos oficiales de MATLAB deben consultarse para la distribución Linux utilizada.

## Rutas de trabajo

En los comandos Linux de esta guía se pueden definir:

```
MATLAB_ROOT="$HOME/Matlab"
REPO_ROOT="$HOME/taller_embebidos/mini-taller-AutoCodeGenMATLAB"
cd "$REPO_ROOT"
```

Son las rutas de este equipo. En otra computadora hay que adaptarlas. Usar la ruta completa del ejecutable evita depender de que `matlab` esté en `PATH`. Los bloques de código `bash` se ejecutan en un terminal Linux; los de `matlab`, en la Command Window de MATLAB.

## 3. Preparar la instalación base de MATLAB

**Objetivo:** disponer de MATLAB R2026b funcionando antes de añadir productos. **Estado:** procedimiento de referencia para otro equipo; la instalación base de este equipo ya funcionaba y no se reinstaló.

1. Acceder a las descargas oficiales de MathWorks con la cuenta autorizada. Seleccionar R2026b para Linux y comprobar los requisitos del sistema.
2. Descargar y descomprimir el instalador en una carpeta de usuario. Conservar la estructura completa del instalador, no solo su ejecutable.
3. Desde la carpeta extraída, iniciar el instalador mediante `./install`. Completar la autenticación y la selección de licencia directamente en su interfaz. No copiar credenciales en scripts ni en capturas para la defensa.
4. Elegir una carpeta de instalación con permiso de escritura. En este equipo se utiliza `/home/dlynchr/Matlab`. No usar privilegios de administrador si no son necesarios para el destino elegido.
5. Instalar MATLAB y seguir las indicaciones de activación. Si se seleccionan otros productos aquí, comprobar cuáles quedaron instalados antes del paso 4.
6. Abrir MATLAB y comprobar que inicia. No borrar una instalación anterior que funcione para repetir este procedimiento.

En este equipo se conservaba el instalador bajo `/home/dlynchr/Matlab/matlab_R2026b_Linux`, mientras que el programa instalado se ejecuta con `/home/dlynchr/Matlab/bin/matlab`: son ubicaciones diferentes.

**Resultado de entrada realmente observado:** MATLAB iniciaba, pero solo registraba MATLAB y MATLAB Copilot. Había carpetas de otros productos que no constituían una instalación funcional.

## 4. Diagnosticar y completar los productos MathWorks

### 4.1. Consultar el entorno sin exponer la licencia

Con el repositorio como carpeta actual de MATLAB:

```
addpath(fullfile(pwd, 'scripts'));
estado = verificar_entorno();
```

**Qué ocurre:** `addpath` permite encontrar nuestras funciones. El diagnóstico consulta `ver` con salida, enumera productos, busca la entrada de Simulink con `which` y lista complementos. Devuelve la estructura `estado` para consultar indicadores sin interpretar texto impreso.

**Qué no demuestra:** que una licencia se pueda utilizar para cualquier operación, que el soporte Arduino compile o que el firmware funcione. Son comprobaciones posteriores.

### 4.2. Obtener MATLAB Package Manager

Guardar el trabajo y cerrar MATLAB antes de modificar la instalación. En un terminal Linux, comprobar que están disponibles `curl` y `unzip` y obtener `mpm` del sitio oficial:

```
command -v curl
command -v unzip
MPM_DIR=$(mktemp -d /tmp/minitaller-mpm.XXXXXX)
curl --fail --location --silent --show-error \
  https://www.mathworks.com/mpm/glnxa64/mpm \
  --output "$MPM_DIR/mpm"
chmod u+x "$MPM_DIR/mpm"
"$MPM_DIR/mpm" --version
"$MPM_DIR/mpm" install --help
```

**Por qué:** `mpm` permite añadir productos de forma reproducible, resuelve dependencias y omite productos ya instalados. La carpeta temporal evita guardar el instalador en Git. `chmod` permite ejecutarlo al usuario que lo descargó.

Si falta una herramienta o la descarga falla, resolver ese prerrequisito antes de continuar. `MPM_DIR` pertenece a ese terminal; en uno nuevo se debe recuperar la ruta válida o repetir la descarga, no reutilizar ciegamente una ruta temporal.

### 4.3. Añadir los productos utilizados

En el mismo terminal, con `MATLAB_ROOT` definido según el paso 2:

```
"$MPM_DIR/mpm" install --release=R2026b \
  --destination="$MATLAB_ROOT" \
  --products=Simulink Simulink_Coder Embedded_Coder
```

| Producto | Papel en este trabajo |
|---|---|
| MATLAB | Entorno que ejecuta los scripts y administra el proyecto |
| Simulink | Editor y motor del modelo de bloques |
| Simulink Coder | Generación de código a partir del modelo |
| Embedded Coder | Target ERT y capacidades orientadas a código embebido |
| MATLAB Coder | Dependencia añadida automáticamente por el instalador |

La elección corresponde al flujo de generación y trazabilidad de este taller. No implica que todos esos productos sean obligatorios para cualquier operación con Arduino. Instalar software tampoco concede una licencia para utilizarlo.

**Resultado observado:** `Installation complete`; los cuatro productos añadidos aparecieron con versión 26.2. No se necesitó `sudo`.

### 4.4. Verificar que Simulink funciona

Reabrir MATLAB, volver a ejecutar el diagnóstico y probar un modelo temporal con un nombre que no esté abierto ni se use para otro trabajo:

```
estado = verificar_entorno();
assert(estado.simulinkDisponible);
assert(estado.simulinkCoderInstalado);
assert(estado.embeddedCoderInstalado);
new_system('verificacion_instalacion_minitaller');
load_system('verificacion_instalacion_minitaller');
close_system('verificacion_instalacion_minitaller', 0);
```

**Qué ocurre:** se crea, carga y descarta un modelo vacío en memoria. No se guarda un archivo ni se programa hardware. Esta prueba básica pasó; el uso del generador y el compilador se comprobó más adelante al construir el firmware.

## 5. Instalar y comprobar el soporte Arduino

**Objetivo:** añadir los bloques de E/S y las herramientas específicas de la placa. Cerrar MATLAB nuevamente y ejecutar en el terminal que contiene `MPM_DIR`:

```
"$MPM_DIR/mpm" install --release=R2026b \
  --destination="$MATLAB_ROOT" \
  --products=Simulink_Support_Package_for_Arduino_Hardware
```

Se instaló la versión 26.2.3 en `/home/dlynchr/Documents/MATLAB/SupportPackages/R2026b`. No confundir este paquete con **MATLAB Support Package for Arduino Hardware**, orientado a interacción desde MATLAB: aquí se utiliza el soporte de **Simulink** para generar firmware.

Reabrir MATLAB y verificar la biblioteca:

```
which('arduinolib')
load_system('arduinolib');
find_system('arduinolib', 'SearchDepth', 2, ...
    'RegExp', 'on', 'Name', 'Analog Input|PWM')
close_system('arduinolib', 0);
```

**Resultado observado:** se encontraron `arduinolib/Analog Input` y `arduinolib/PWM`. La instalación incluyó herramientas AVR que después lograron compilar y enlazar. Para el flujo virtual no fue necesario conectar una placa; la configuración USB del hardware real se verificará al preparar la demo física.

## 6. Entender y crear el modelo

### 6.1. El diagrama representa un algoritmo

Las líneas de Simulink transportan valores entre bloques; no son los cables eléctricos del montaje. El modelo se encuentra en ../model/pot_led.slx.

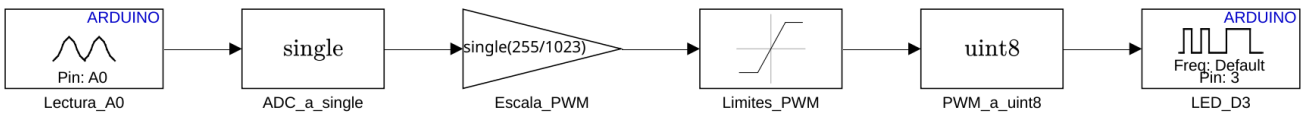



*Figura 1. Exportación del modelo real incluido en el repositorio, sin recompilar ni guardar cambios en él. Los símbolos de los bloques no son capturas de señales ni mediciones de frecuencia.*

| Bloque, en orden | Función y motivo |
|---|---|
| `Lectura_A0` | Lee A0 cada 0.01 s; entrega un `uint16` con el resultado ADC del Uno, de 0 a 1023 |
| `ADC_a_single` | Convierte a precisión simple para realizar el escalamiento fraccionario |
| `Escala_PWM` | Multiplica por `single(255/1023)`, adaptando el rango ADC al mando PWM de 8 bits |
| `Limites_PWM` | Restringe la señal al intervalo 0–255 |
| `PWM_a_uint8` | Convierte a entero de 8 bits con Floor y saturación; descarta la fracción para valores no negativos |
| `LED_D3` | Aplica el mando al PWM del pin D3 usando la frecuencia predeterminada del soporte |

Ejemplos conceptuales del escalamiento, no mediciones capturadas del simulador:

| Lectura ADC | Mando PWM esperado | Interpretación |
|---|---|---|
| 0 | 0 | Salida mínima, LED apagado |
| 512 | 127 | Mando aproximadamente a la mitad de su rango |
| 1023 | 255 | Salida máxima |

El valor 127 proviene de truncar el resultado fraccionario, no de redondearlo al entero más cercano. El brillo percibido no es necesariamente lineal respecto al mando PWM; verlo cambiar no mide exactamente el ciclo de trabajo.

## 6.2. Configuración temporal y de generación

- Placa: **Arduino Uno**, correspondiente al R3 con ATmega328P.
- Solver: **Fixed-step**, **FixedStepDiscrete**, paso **0.01 s**.
- Generación: target **ert.tlc**, lenguaje C y reporte habilitado.
- Acción del target: **Build**, sin carga automática a hardware.

El paso de 10 ms configura la actualización del algoritmo a 100 actualizaciones por segundo. **No fija la frecuencia portadora del PWM**, producida por el temporizador del microcontrolador. Ninguna de esas temporizaciones se ha medido todavía. El `StopTime = 1` guardado es un ajuste de simulación; no debe interpretarse como una instrucción para apagar el LED al segundo de ejecución.

## 6.3. Crear el modelo desde cero, solo cuando sea necesario

El repositorio ya incluye el modelo. Para inspeccionarlo o compilarlo, omitir este paso. En una copia separada de trabajo donde aún no exista el archivo:

```
addpath(fullfile(pwd, 'scripts'));
rutaModelo = preparar_modelo();
```

El script resuelve las rutas desde su ubicación, carga la biblioteca Arduino, crea el modelo, configura los bloques, conecta sus puertos y guarda el archivo. Las coordenadas `Position` solo determinan el dibujo del diagrama.

Se niega a sobrescribir un modelo existente o a reemplazar uno abierto. Si aparece `minitaller:ModeloExistente`, no es un fallo de instalación: abrir el modelo incluido o usar otra copia de trabajo, conservando el original.

# 7. Generar y compilar sin una placa física

## 7.1. La diferencia entre generar, compilar y desplegar

1. **Generar:** transformar el diagrama y sus parámetros en código fuente.
2. **Compilar y enlazar:** transformar las fuentes y dependencias en un programa completo para el ATmega328P, no para el procesador Linux.
3. **Desplegar:** transferir ese programa a una placa. Este paso se excluye del tutorial virtual; Wokwi carga el archivo preparado.

El build utiliza `avr-gcc` para C y `avr-g++` para C++. Además del modelo, participan controladores, runtime, scheduler y core Arduino. Por eso compilar únicamente los archivos C del modelo no reproduce el enlace completo.

## 7.2. Problema encontrado y corrección

La primera llamada a `slbuild` terminó intentando una descarga y falló con `No Arduino Uno board detected`. El target tenía:

```
Runtime.BuildAction = 'Build, load and run'
```

La selección `StandaloneCoderTarget` por sí sola no desactivó ese hook del target. Se corrigió usando el objeto de configuración del modelo:

```
configuracion = getActiveConfigSet('pot_led');
codertarget.data.setParameterValue(configuracion, ...
    'Runtime.BuildAction', 'Build');
```

Esta corrección ya está incorporada al generador, al modelo guardado y al script de compilación, que además verifica el valor antes de construir. No conectar una placa para ocultar este fallo ni usar **Build, Deploy & Start** en el flujo virtual. La API y el comportamiento se comprobaron en R2026b.

## 7.3. Ejecutar la compilación reproducible

Guardar y cerrar el modelo. Con el repositorio como carpeta actual de MATLAB:

```
addpath(fullfile(pwd, 'scripts'));
resultado = compilar_modelo();
```

Alternativa desde Linux, con las variables del paso 2:

```
cd "$REPO_ROOT"
"$MATLAB_ROOT/bin/matlab" -batch \
  "addpath(fullfile(pwd, 'scripts')); resultado = compilar_modelo();"
```

**Qué hace el script, en orden:**

1. Comprueba que existe el modelo y no está abierto, para no descartar cambios.
2. Define la carpeta `build` y una caché separada; conserva la configuración anterior para restaurarla al salir mediante `onCleanup`.
3. Carga la versión guardada, comprueba la placa y fuerza la acción `Build`.
4. Invoca `slbuild` para generar y compilar, no solamente producir fuentes.
5. Busca un único HEX y un único ELF. Si hay archivos ambiguos o faltantes, se detiene en lugar de elegir uno silenciosamente.
6. Copia los binarios a `firmware` y devuelve sus rutas. Una recompilación reemplaza esas copias y obliga a repetir las pruebas del simulador.

**Criterio de éxito observado:** `Successful completion of build procedure for: pot_led`, sin búsqueda de una placa, seguido de las rutas de ambos artefactos. Si una nueva compilación falla, las copias anteriores pueden seguir existiendo: su presencia no demuestra que el último intento haya terminado correctamente.

### 7.4. Qué contienen los resultados

| Resultado | Uso |
|---|---|
| HEX | Imagen de programa en formato Intel HEX que el simulador puede cargar |
| ELF | Ejecutable con secciones y símbolos; útil para herramientas de análisis y depuración |
| C generado localmente | Permite revisar cómo los bloques se traducen a operaciones y llamadas a controladores |
| Reporte local de generación | Facilita inspeccionar la generación; no demuestra validación funcional |

El C generado localmente se encuentra bajo `build/pot_led_ert_rtw`. En `pot_led.c`, la función `pot_led_step` lee A0, escala, limita, convierte y llama al controlador PWM. Los comentarios referencian nombres como `Lectura_A0` y `LED_D3`. El identificador Arduino 14 que aparece para A0 no indica un cable adicional. Esta revisión se hizo sobre el código producido por Simulink, no sobre un sketch C escrito manualmente en su lugar.

## 8. Configurar el circuito y ejecutar Wokwi

### 8.1. Instalar y activar la extensión

Abrir el repositorio completo en VS Code. Instalar **Wokwi Simulator**, del editor Wokwi, desde Extensions o ejecutar:

```
code --install-extension wokwi.wokwi-vscode
```

Se instaló la versión 4.0.0. Después, usar `F1` → **Wokwi: Request a new License** y completar el proceso directamente en navegador y VS Code. Si aparecen condiciones de pago, revisarlas antes de continuar; no se requiere compartir claves en esta guía. Si los comandos no aparecen, guardar el trabajo y usar **Developer: Reload Window**.

## 8.2. Comprender el montaje virtual

| Conexión | Propósito |
|---|---|
| Uno 5V → potenciómetro VCC | Alimentación de referencia del potenciómetro virtual |
| Uno GND → potenciómetro GND | Referencia común |
| Potenciómetro SIG → Uno A0 | Entrada analógica que lee el modelo |
| Uno D3 → resistencia de 330 ohmios → ánodo LED | Rama de salida PWM |
| Cátodo LED → Uno GND | Retorno de la rama del LED |

La resistencia limita corriente en un montaje real; aquí representa la rama que se trasladará a la demo. Antes de cablear físicamente habrá que verificar polaridad, valor real, características del LED y límites de la placa. La emulación actual no valida esa seguridad eléctrica.

## 8.3. Comprender los dos archivos Wokwi

En ../wokwi_simulation/diagram.json:

- `parts` enumera Uno, potenciómetro, resistencia y LED, con IDs únicos.
- `attrs` configura valores como 330 ohmios, color rojo y posición inicial del potenciómetro. `top` y `left` solo ubican los componentes en pantalla.
- `connections` une terminales por ID y nombre de pin. El color de un cable es una ayuda visual, no una propiedad eléctrica.
- El JSON no contiene firmware ni ejecuta la generación de código.

En ../wokwi_simulation/wokwi.toml:

```
[wokwi]
version = 1
firmware = "../firmware/pot_led.hex"
elf = "../firmware/pot_led.elf"
```

Las rutas se resuelven desde la carpeta del TOML. `..` sube a la raíz del repositorio. Wokwi utiliza los binarios que ya existen: esta configuración no invoca MATLAB ni compila automáticamente el modelo.

## 8.4. Ejecutar y comprobar

1. Detener una simulación anterior antes de cambiar el firmware que se probará.
2. En `F1`, ejecutar **Wokwi: Select Config File** y elegir el TOML de la subcarpeta `wokwi_simulation`.
3. En `F1`, ejecutar **Wokwi: Start Simulator**. Comprobar que aparece el circuito.
4. Recorrer mínimo, posición intermedia y máximo del potenciómetro; volver al mínimo, detener e iniciar nuevamente el simulador.

| Caso | Resultado confirmado por el expositor |
|---|---|
| Mínimo | LED apagado |
| Posición intermedia | Brillo intermedio |

| Caso | Resultado confirmado por el expositor |
|---|---|
| Máximo | Mayor brillo observado |
| Retorno al mínimo | El LED vuelve a apagarse |
| Detener y reiniciar | Se repite el comportamiento |

Estas cinco confirmaciones se registraron el 12 de octubre. Son evidencia manual, no capturas de osciloscopio ni pruebas automáticas. No hay salida UART en el firmware actual: un Monitor Serie vacío no indica por sí solo un fallo.

## 9. Abrir y recorrer el diagrama en Simulink

Este es el último paso de inspección explicado: visualizar el modelo ya creado, sin regenerarlo. Desde la raíz del repositorio en MATLAB:

```
open_system(fullfile(pwd, 'model', 'pot_led.slx'));
set_param('pot_led', 'ZoomFactor', 'FitSystem');
```

1. Leer los bloques de izquierda a derecha siguiendo el paso 6.
2. Abrir `Lectura_A0` con doble clic y mostrar pin 0, tipo `uint16` y muestreo.
3. Abrir `Escala_PWM` y explicar la ganancia `single(255/1023)`.
4. Abrir `PWM_a_uint8` y justificar Floor y saturación.
5. Abrir `LED_D3` y mostrar el pin 3.
6. En **Model Settings**, revisar solver y placa. No cambiar opciones solo para inspeccionarlas. Si se edita algo, guardar deliberadamente y recompilar antes de atribuir el resultado de Wokwi a esa nueva versión.

Abrir el modelo no programa una placa. **Run de Simulink no equivale a ejecutar este circuito en Wokwi.** No está preparada una simulación interna de la planta analógica ni una conexión del Scope de Simulink con el firmware emulado.

## 10. Mapa de archivos y responsabilidades

Los nombres siguientes son enlaces relativos a esta guía. En una copia impresa, localizarlos desde la raíz del repositorio; `../` significa subir desde `docs`.

| Archivo | Responsabilidad |
|---|---|
| ../README.md | Punto de entrada y pasos rápidos de ejecución virtual |
| guia-provisional.md | Recorrido pedagógico y material de avance para esta defensa |
| guia-provisional.pdf | Versión imprimible de la guía; regenerar cuando cambie la fuente |
| figuras/modelo-pot-led.png | Imagen exportada del modelo real para explicar los bloques |

| Archivo | Responsabilidad |
|---|---|
| bitacora.md | Registro histórico: comandos, fallos, correcciones, versiones y resultados |
| ../scripts/verificar_entorno.m | Diagnostica productos y complementos; no instala ni compila |
| ../scripts/preparar_modelo.m | Construye y guarda el diagrama; no sobrescribe un modelo existente |
| ../scripts/compilar_modelo.m | Genera, compila, verifica artefactos y copia firmware; no despliega |
| ../scripts/exportar_guia.py | Convierte esta guía Markdown a PDF; no modifica modelo ni firmware |
| ../model/pot_led.slx | Modelo editable que realmente se compila |
| ../firmware/pot_led.hex | Imagen de programa utilizada por Wokwi |
| ../firmware/pot_led.elf | Ejecutable asociado al mismo build |
| ../wokwi_simulation/diagram.json | Componentes y conexiones virtuales |
| ../wokwi_simulation/wokwi.toml | Rutas que relacionan el simulador con los binarios |
| ../.gitignore | Excluye `build`, cachés `.slxc` y autoguardados `.slx.autosave` de Git |
| ../masterprompt.md | Propuesta original de demo/tutorial; no equivale al estado implementado |
| ../LICENSE | Licencia MIT del repositorio; no reemplaza licencias de terceros |

El instructivo académico original se conserva en ../Instrucciones para trabajo en minitalleres - 2S2026.md. Establece los entregables y la revisión docente, no las opciones de compilación.

La carpeta local `build` contiene código, dependencias de construcción, caché, objetos y reportes generados; no se incluye en Git. El firmware y el modelo sí forman parte de los archivos preparados para compartir. Los scripts están comentados por secciones para relacionar cada operación con esta guía.

No hay todavía una exportación independiente en `generated_code` ni un workflow de GitHub Actions funcional. No presentarlos como archivos ya implementados.

## 11. Evidencias técnicas y sus límites

La medición con `avr-size -C --mcu=atmega328p` sobre el ELF produjo:

| Recurso | Resultado |
|---|---|
| Programa | 4780 bytes, 14.6% de los 32768 bytes del microcontrolador |
| Datos estáticos | 287 bytes, 14.0% de los 2048 bytes de SRAM |

El porcentaje de programa no descuenta una reserva de bootloader. El de datos no mide el máximo de pila ni la memoria dinámica durante ejecución.

La bitácora registra SHA-256 de ambos binarios. Permiten identificar qué archivos se probaron, pero no demuestran por sí mismos que el programa sea correcto. Para volver a obtenerlos desde la raíz del repositorio:

```
sha256sum firmware/pot_led.hex firmware/pot_led.elf
```

**Licencias:** el código generado contiene una cabecera Academic License. No se han eliminado avisos ni se considera que la licencia MIT del repositorio otorgue derechos adicionales sobre MATLAB, Wokwi, runtime o dependencias. La exportación para CI requerirá revisar los componentes que sea posible distribuir.

## 12. Solución de problemas observados o previstos

| Síntoma | Causa o comprobación | Acción |
|---|---|---|
| `matlab` no se encuentra en el terminal | La instalación no está en `PATH`; observado | Usar `"$MATLAB_ROOT/bin/matlab"` |
| Hay carpetas, pero Simulink no aparece | Instalación incompleta; observado | Diagnosticar con `verificar_entorno`, instalar y volver a probar |
| No se encuentra `arduinolib` | Soporte ausente o no registrado; caso preventivo | Revisar complementos y su versión; no sustituirlo por el paquete MATLAB de Arduino |
| `ModeloExistente` o `ModeloAbierto` | Protección deliberada de los scripts | Abrir el modelo incluido o guardar/cerrar; no borrar trabajo para superar el control |
| `No Arduino Uno board detected` al compilar | Acción `Build, load and run`; observado | Usar el script corregido, que fuerza y verifica `Build` |
| Wokwi no encuentra configuración o firmware | Configuración anidada o ruta incorrecta; caso preventivo | Seleccionar el TOML y verificar sus rutas relativas y el build exitoso |
| La activación de Wokwi impide iniciar | Acceso no disponible; caso preventivo | Revisar cuenta y condiciones en su interfaz, sin compartir claves |

No describir como observados los casos preventivos. Si cambia el entorno, registrar el mensaje exacto y la solución que se haya comprobado en la bitácora.

## 13. Recorrido sugerido para la primera defensa

Este guion breve es una propuesta para explicar el avance, no el cronograma final del mini-taller de 90–110 minutos.

| Orden | Qué mostrar | Qué explicar |
|---|---|---|
| 1 | Estado de avance de esta guía | Tutorial virtual ahora; demo física y CI después |

| Orden | Qué mostrar | Qué explicar |
|---|---|---|
| 2 | Modelo abierto en Simulink | Lectura, escalamiento, conversión y salida |
| 3 | Scripts comentados | Preparación reproducible y protección de trabajo existente |
| 4 | Acción `Build` y artefactos | Diferencia entre generar, enlazar y desplegar |
| 5 | Circuito Wokwi | Firmware generado controlando un Uno emulado |
| 6 | Pruebas y pendientes | Qué se observó y qué todavía no puede afirmarse |

### Preguntas técnicas para preparar la explicación

- **¿Por qué convertir de 0–1023 a 0–255?** El ADC del Uno entrega 10 bits y el mando de este bloque PWM utiliza 8 bits.
- **¿Por qué no basta dividir con enteros?** El escalamiento necesita conservar la fracción hasta la conversión final; por eso se utiliza `single`.
- **¿El PWM es de 100 Hz?** No se deduce del paso de 10 ms. El muestreo del algoritmo y la portadora PWM son conceptos distintos; falta medirlos.
- **¿Qué demuestra que no hace falta una placa para el tutorial?** El build corregido y la ejecución en Wokwi se realizaron sin descargar a hardware.
- **¿Ya hay CI/CD?** No. Hay un flujo local reproducible; la receta independiente, el análisis estático y GitHub Actions son el siguiente trabajo de integración.

Estas preguntas apoyan la defensa; no sustituyen el instrumento diagnóstico de cinco preguntas que exige el instructivo para el taller final.

### Lista de comprobación antes de la reunión

- [ ] Abrir la guía y el modelo antes de compartir pantalla.
- [ ] Comprobar acceso a Wokwi e Internet; ensayar las cinco pruebas.
- [ ] Conservar el firmware probado; no regenerar a última hora sin repetirlas.
- [ ] Identificar en el código generado las operaciones de `pot_led_step`.
- [ ] Llevar claros los pendientes y el estado real de NotebookLM y sus fuentes.
- [ ] Confirmar con el profesor el calendario y los criterios de aprobación.

La presentación está prevista para el 19 de octubre y la primera revisión para el 13. El instructivo exige revisión una semana antes, por lo que corresponde confirmar esa diferencia con el profesor. Esta guía de avance no equivale a su aprobación ni a los seis entregables finales completos.

## 14. Trabajo restante

1. Decidir y añadir instrumentación UART y medición de PWM; volver a compilar y probar. No afirmar que el Scope de Simulink ya recibe datos de Wokwi.

2. Preparar una exportación reproducible con sus dependencias y licencias; configurar análisis y build en GitHub Actions.
3. Probar la demo física con componentes verificados y criterios equivalentes.
4. Completar el proceso en NotebookLM: fuentes técnicas de calidad, preguntas, citas, síntesis crítica y acceso para el profesor.
5. Producir presentación, tutorial PDF final, diagnóstico, código/configuraciones y reflexión sobre NotebookLM; ensayar y preparar contingencias.

Para la entrega final, aplicar la nomenclatura `Apellido_Tema_TipoDocumento.pdf` cuando se complete la identificación académica. El nombre provisional de esta guía no pretende sustituir esa convención.

## 15. Referencias y procedencia

- Requisitos académicos: instructivo del curso conservado en el repositorio.
- Ejecución y resultados: bitacora.md, scripts, modelo y firmware de este repositorio; confirmaciones manuales del expositor para Wokwi.
- MathWorks, Descargas de productos y requisitos del sistema: referencias para la instalación inicial, no evidencia de que se haya repetido.
- MathWorks, Get MATLAB Package Manager.
- MathWorks, Install Support Packages Noninteractively.
- MathWorks, Install Support for Arduino Hardware.
- MathWorks, slbuild.
- Wokwi, Getting Started y Project Configuration.

Esta guía fue preparada con asistencia de GitHub Copilot a partir de los archivos y resultados de trabajo. No se atribuye a NotebookLM contenido que todavía no haya sido generado y validado allí.

## 16. Volver a exportar esta guía a PDF

El Markdown es la fuente editable. El exportador usa Python 3, el paquete Python-Markdown 3.7 y Google Chrome instalado. Desde la raíz del repositorio:

```
python3 -m venv build/pdf-venv
build/pdf-venv/bin/python -m pip install Markdown==3.7
build/pdf-venv/bin/python scripts/exportar_guia.py
```

La dependencia y el HTML intermedio quedan en `build`, excluido de Git. El PDF se genera junto a esta guía, con el mismo nombre base. El navegador utiliza un perfil temporal, sin leer la sesión personal del expositor.

El PDF es para presentar e imprimir; el Markdown conserva la edición y los enlaces locales al repositorio. Una copia aislada del PDF no incluye los scripts, el modelo o el firmware enlazados. Revisar las páginas después de modificar la guía; esta exportación no ejecuta ni repite las pruebas del sistema embebido.